Research Paper Summaries and Relevance to the Project

COS40005 Computing Technology Project A · VIB Internal Knowledge Assistant ·

Oct 31, 2026 · @Truong Ngoc Gia Hieu

Ten papers from the project's Research Paper folder were read in full text and summarised below. Only two of them (Jeong & Lee 2025; Bhatt et al. 2025) address access control in RAG, which is the project's distinguishing requirement; the others supply foundations, infrastructure and evaluation practice.

1. Overview

1.1 Scope and method

Each paper was read from the PDF in the project folder. For each, the summary records the purpose, method, dataset and evaluation, key results and stated limitations, followed by an assessment of relevance to the VIB Internal Knowledge Assistant (a Slack bot that answers from Confluence with citations and permission-aware retrieval). Statements marked *inference* are the author's reading, not claims made by the paper.

1.2 Relevance scale

- **High:** the paper directly supports a design decision or the evaluation plan.
- **Medium:** the paper gives useful background or a comparison, but the system or domain differs.
- **Low:** the paper is related in topic only and can be cited for contrast.

1.3 The ten papers at a glance

#	Paper	Type	Relevance	Main use in the project
1	Lewis et al. (2020)	Conference paper (NeurIPS)	High	Defines RAG; HNSW-based dense retrieval; motivates provenance and updatable knowledge
2	Gao et al. (2024)	Survey (arXiv)	High	Taxonomy (naive, advanced, modular RAG) and evaluation vocabulary

#	Paper	Type	Relevance	Main use in the project
3	Jeong & Lee (2025)	Journal (IEEE Access)	High	Closest work on permission-aware RAG; contrast with our pre-filtering design
4	Bhatt et al. (2025)	Position paper (arXiv)	High	Argues for deterministic access control; the participant principle
5	Zhou et al. (2025)	Preprint (arXiv)	Low	Encryption alternative to metadata filtering, for contrast
6	Mukherjee et al. (2025)	Industry paper (FSE 2025 Companion)	Medium	Enterprise documentation assistant with expert evaluation
7	Es et al. (2024)	Demo paper (EACL)	High	RAG evaluation metrics (faithfulness, relevance) for Section 8
8	Boonsri & Buranasaksee (2025)	Conference paper (IEEE InCIT)	Medium	Layered architecture, adapters, acceptance testing
9	Mohammed & Aref (n.d.)	Overview paper	Low	Vocabulary (NLU, DM, NLG); component checklist
10	Malkov & Yashunin (2020)	Journal (IEEE TPAMI)	Medium	HNSW index used by pgvector (Layer 4)

2. RAG Foundations

2.1 Lewis et al. (2020): Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Aspect	Summary
Reference	Lewis, P et al. 2020, NeurIPS 2020 (arXiv:2005.11401), Facebook AI Research, University College London and New York University

Aspect	Summary
Purpose and topic	Large language models store facts in their parameters, cannot easily update them and cannot show where an answer came from. The paper introduces retrieval-augmented generation (RAG), which combines a pre-trained generator (parametric memory) with a searchable document index (non-parametric memory), and tests it on knowledge-intensive tasks.
Method	Retriever: a Dense Passage Retrieval (DPR) bi-encoder using two BERT-base encoders, with top-k chosen by maximum inner product search. Generator: BART-large (about 400 million parameters), which receives the question concatenated with each retrieved passage. Two variants: RAG-Sequence uses the same retrieved passages for the whole answer, RAG-Token can use different passages for each generated token. The query encoder and the generator are fine-tuned together, while the document encoder and the index are kept fixed.
Dataset and evaluation	Knowledge source: one Wikipedia dump (December 2018), split into disjoint 100-word chunks, giving about 21 million documents, indexed with FAISS using a Hierarchical Navigable Small World approximation. Tasks: open-domain question answering (Natural Questions, TriviaQA, WebQuestions, CuratedTrec), abstractive QA (MS-MARCO), Jeopardy question generation, and fact verification (FEVER). Metrics: exact match for QA, ROUGE and BLEU for generation, label accuracy for FEVER, plus human judgements of factuality and specificity.

Aspect	Summary
Key results	State of the art on open-domain QA at the time (for RAG-Sequence: 44.5 on Natural Questions, 56.8 on TriviaQA, 45.2 on WebQuestions, 52.2 on CuratedTrec), without needing a separate re-ranker or extractive reader. In human evaluation of Jeopardy question generation, RAG was judged more factual than BART in 42.7% of cases against 7.1% for BART, and more specific in 37.4% against 16.8%. The authors also show that replacing the index (a 2016 against a 2018 Wikipedia dump) updates the model's knowledge without retraining.
Limitations	Evidence comes from Wikipedia and public benchmarks, not private enterprise documents. The paper does not discuss access control or per-user permissions (no mention in the text).
Relevance to the project	**High.** This is the foundation of the system: retrieval first, generation second, with an index that can be updated. It supports the project's use of HNSW-based vector search (Layer 4) and the argument that updating documents does not require retraining. Our design differs in two ways: it uses a frozen LLM with prompt-level grounding instead of fine-tuning, and it adds permission filtering, which the paper does not address.

2.2 Gao et al. (2024): Retrieval-Augmented Generation for Large Language Models: A Survey

Aspect	Summary
Reference	Gao, Y et al. 2024, arXiv:2312.10997v5 (27 March 2024), Tongji University and Fudan University
Purpose and topic	A systematic review of RAG for large language models, covering how the field developed after ChatGPT, the main techniques, and how RAG systems are evaluated.

Aspect	Summary
Method	Literature survey of more than 100 RAG studies. It sorts methods into three paradigms and analyses three core stages: retrieval, generation and augmentation. Naive RAG follows indexing, retrieval and generation (the "retrieve-read" pattern: clean and chunk documents, embed them, retrieve the top-k chunks, then generate). Advanced RAG adds optimisation before retrieval (for example query rewriting) and after it (for example re-ranking). Modular RAG treats components as interchangeable modules with flexible flows.
Dataset and evaluation	No new dataset. It catalogues downstream tasks and datasets (for example Natural Questions, TriviaQA, SQuAD, WebQuestions, PopQA and MS MARCO) and evaluation practice: retrieval metrics (Hit Rate, MRR, NDCG), three quality scores (context relevance, answer faithfulness, answer relevance) and four abilities (noise robustness, negative rejection, information integration, counterfactual robustness). Benchmarks listed: RGB, RECALL, CRUD; automated tools: RAGAS, ARES, TruLens.
Key results	A taxonomy and an evaluation framework rather than experimental findings. It identifies open challenges such as RAG against long-context models, robustness, and engineering concerns including data security.
Limitations	As a survey it reports no experiments. Security is raised only briefly as an engineering challenge (for example preventing inadvertent disclosure by the model); access control is not developed.
Relevance to the project	**High.** The project is a Naive RAG pipeline with two Advanced RAG additions planned for Beta (re-ranker and conversation memory). The survey's term "negative rejection" describes exactly the fallback behaviour the project requires when no relevant passage exists, and its quality scores can be used directly in the evaluation plan.

3. Access Control in RAG

3.1 Jeong & Lee (2025): Permission-Aware RAG

Aspect	Summary
Reference	Jeong, J & Lee, S-G 2025, IEEE Access, vol. 13, pp. 192819-192835, doi:10.1109/ACCESS.2025.3628960, Seoul National University. Open-access (CC BY 4.0), code released on GitHub.
Purpose and topic	Most RAG research assumes every user may read every retrieved document, which breaks down in enterprises where data sits in several systems with different access policies. The paper proposes a permission-aware RAG framework that checks each retrieved document against the data owner's own Identity and Access Management (IAM) system in real time, without merging policies into one.
Method	Index time: each resource receives a unique resourceID; chunks, embeddings and resourceID go into a central vector store, and IAM metadata goes into a metadata store. Query time: embed the question, retrieve the top-k documents, then a Permission Retriever asks each provider's IAM (GCP IAM, AWS IAM or Keycloak) whether the user may read each document, using a Credential Manager and an IAM Adapter, and passes only authorised documents to the LLM. Threat model: Zero Trust, with information disclosure and elevation of privilege in scope; spoofing, tampering, repudiation and denial of service out of scope.

Aspect	Summary
Dataset and evaluation	HotpotQA development set (distractor setting): 13,783 supporting documents and 7,405 multi-hop questions. Embeddings: all-MiniLM-L6-v2 (384 dimensions) in a FAISS store; answers by gpt-4o-mini. Three experiments: (1) scenario-based, with 13 user profiles and 20 questions each over three services (Film on GCP RBAC, Music on AWS ABAC, People on Keycloak RBAC), compared with a control group that merges policies into one role; (2) large-scale, with 1,000 random questions, k=5, eight user profiles, measuring a Permission Coverage metric against exact match and F1; (3) latency, varying top-k and worker concurrency.
Key results	The control group leaked data because a merged role dropped the country=US condition, which shows the risk of manual policy merging. The delegated IAM approach enforced the expected allow and deny decisions. Answer quality falls as permission coverage falls, which the paper quantifies as the security and quality trade-off. Latency: raising top-k from 1 to 20 added about 140 ms of IAM latency, small against LLM inference; raising workers from 1 to 15 cut mean IAM latency by about 70 percent (from 420.71 ms to 127.33 ms). Caching of permission checks was switched off during measurement.
Limitations	Depends on the availability of external IAM services. HotpotQA (Wikipedia) does not capture real enterprise scale or policy complexity. Authors plan TTL caching, health checks with fallback, better retrieval when permission coverage is low, and encryption of embeddings and metadata.

Aspect	Summary
Relevance to the project	**High, and the closest paper to our contribution.** It confirms the problem and the need for per-user, real-time authorisation. Our design differs in an important way (inference): the paper filters after retrieving the top-k, so a user with little access may receive few or no usable documents, which is why answer quality drops. Our design places the access filter inside the vector query (space, department, access level not above the tier), so the top-k is chosen only from permitted text. Our design also uses a simpler tier model held in one database rather than live calls to several IAM systems, which suits one Confluence source but would not scale to many independently governed sources. Its metrics (permission coverage, exact match, F1, IAM latency) can be reused in our evaluation.

3.2 Bhatt et al. (2025): Enterprise AI Must Enforce Participant-Aware Access Control

Aspect	Summary
Reference	Bhatt, SS et al. 2025, arXiv:2509.14608 (position paper, preprint), Microsoft
Purpose and topic	Argues that fine-tuned models and RAG pipelines in enterprises can leak confidential data to people without permission, and that probabilistic defences (prompt sanitisation, output filtering, system isolation, training-level privacy) cannot guarantee protection. It takes the position that only deterministic, fine-grained access control, enforced at fine-tuning and at inference, is reliable.

Aspect	Summary
Method	Conceptual and security analysis rather than an experiment. Section 2 demonstrates data-extraction attacks on fine-tuned models. Section 3 proposes access-control-aware fine-tuning: a user may use a fine-tuned model only if authorised for all training data, and choosing data and users is formulated as finding bicliques in a bipartite graph. Section 4 shows a cross-prompt injection attack (XPIA) on an email assistant that uses RAG, where a malicious sender hides instructions to make the assistant leak the other user's data, and explains why existing defences fail. It then states the principle that any content used in training, retrieval or generation must be authorised for all participants in the interaction.
Dataset and evaluation	No dataset or benchmark. Evidence is attack scenarios and argument. The authors report that an initial solution is deployed in Microsoft Copilot Tuning.
Key results	The central claim is that deterministic access control is needed at every stage, and that human review of outputs does not stop the described attack. No quantitative results are reported in the text.
Limitations	It is a position paper without measured evaluation. The authors list open issues: weights on documents, checking access for multiple models, ACLs that change over time, and misconfigured ACLs.
Relevance to the project	**High.** It supports the project's rule that permissions are enforced before the LLM sees text, and not by asking the model to hide information. It also raises a point the project should handle (inference): in Slack, a bot reply in a channel or thread can be read by every member of that channel, so "authorised for all participants" means the answer should not exceed what the channel's readers may see, not only what the asker may see. This is not yet specified in our design and should be listed as an open decision.

3.3 Zhou et al. (2025): Privacy-Aware RAG: Secure and Isolated Knowledge Retrieval

Aspect	Summary
Reference	Zhou, P, Feng, Y & Yang, Z 2025, arXiv:2503.15548, Beijing University of Posts and Telecommunications (preprint)
Purpose and topic	Protects a RAG knowledge base from unauthorised access and leakage (including prompt-injection extraction) by encrypting both text chunks and embeddings before storage.
Method	User authentication and a hierarchical key structure give each user exclusive access to private data. Method A maps primary keys to per-user AES-CBC keys. Method B chains data nodes with dynamically derived keys and hash-based integrity checks for higher security. A secure similarity computation runs on public data, and the system filters out other users′ private data before returning results.
Dataset and evaluation	No dataset and no experiments appear in the paper. Sections are Introduction, Related Work, Methodology, Security Proof and Conclusion. The evidence is a set of security proofs. The claim that performance is preserved is asserted in the abstract and conclusion rather than measured.
Key results	Security proofs of resilience to the threats considered. No quantitative results.
Limitations	Untested in practice; future work mentions scalability to larger datasets. Encryption adds key management and may not fit approximate vector search without further work (inference).

Aspect	Summary
Relevance to the project	**Low.** Useful as a contrast: the project chooses metadata filtering with tiers, which is simpler and sufficient for one organisation, instead of per-user encryption. It can be cited as an option for a hardened production release, together with the encryption plans that Jeong & Lee (2025) list as future work.

4. Enterprise Assistants and Evaluation

4.1 Mukherjee et al. (2025): From Documents to Dialogue: Building KG-RAG Enhanced AI Assistants

Aspect	Summary
Reference	Mukherjee, M, Kim, S, Chen, X, Luo, D, Yu, T & Mai, T 2025, Companion Proceedings of the 33rd ACM Symposium on the Foundations of Software Engineering (FSE '25), Trondheim, Norway, 23-27 June, ACM; arXiv:2502.15237. Carnegie Mellon University and Adobe Research. The copy reviewed carries a placeholder DOI, so cite the arXiv number.
Purpose and topic	Describes how the Adobe Experience Platform AI Assistant answers questions over more than 17,000 pages of proprietary documentation. Plain vector search misses relationships between concepts, so the authors build a Knowledge Graph (KG) from the documents and use it for retrieval (KG-RAG). The contribution is a way of building a high-quality, low-noise KG.

Aspect	Summary
Method	Pipeline: (1) clean the pages (remove HTML, scripts, links, author data) and summarise each with an LLM; (2) discover entities incrementally, giving the LLM the list of entities found so far and rejecting semantic duplicates; (3) extract relations as triplets, each with an LLM-assigned confidence from 0 to 1, keeping only triplets at or above 0.6 (threshold chosen by A/B testing); (4) store the source URL with every relation for provenance. At query time the question is embedded, the top-1 tuple is retrieved from the KG and added to the prompt sent to the LLM (Azure OpenAI).
Dataset and evaluation	100 question-answer pairs on the 'segment' domain, taken from production traffic and rated by Adobe domain experts on a three-point scale (fully relevant, moderately relevant, irrelevant). The baseline is the production system, which retrieves documents by embedding cosine similarity. Evaluation uses an LLM as judge (with in-context examples and chain-of-thought; 85 percent agreement with humans) plus cosine similarity of BERT embeddings between answers.
Key results	Irrelevant answers fell from 52 to 25 (a 51.9 percent reduction); moderately relevant rose from 31 to 53; fully relevant rose from 17 to 32 (an 88.2 percent increase). Average cosine similarity with the fully relevant reference answers was 0.89.
Limitations	Small single-domain dataset; the LLM judge may be biased (it tended to rate more answers as moderately relevant than humans did); scale and latency of graph construction; prompts and code are not released. Only top-1 retrieval was used (top-k is future work). The paper does not describe per-user access control.

Aspect	Summary
Relevance to the project	**Medium.** It is a real enterprise documentation assistant, so it supports the motivation for the project and gives a template for how to report results (a baseline against an improved retriever, expert-rated relevance, LLM-as-judge with a stated agreement rate). The project does not build a knowledge graph; it uses chunked text with vector search, so the technique itself is a possible future extension and not part of the current scope. Its provenance idea (store the source URL with each fact) matches our design of returning Confluence source links with each answer.

4.2 Es et al. (2024): RAGAs: Automated Evaluation of Retrieval Augmented Generation

Aspect	Summary
Reference	Es, S, James, J, Espinosa-Anke, L & Schockaert, S 2024, Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL), pp. 150-158, Association for Computational Linguistics. Exploding Gradients and Cardiff University. Code: github.com/explodinggradients/ragas.
Purpose and topic	RAG systems are hard to evaluate because several things matter at once: whether the retrieved text is relevant, whether the model uses it faithfully, and whether the answer is good. RAGAs offers reference-free metrics, so quality can be estimated without human-written ground-truth answers.

Aspect	Summary
Method	Three metrics, all computed by prompting an LLM (gpt-3.5-turbo-16k). Faithfulness: split the answer into statements and check each against the retrieved context, score = supported statements / all statements. Answer relevance: generate n questions from the answer and average their embedding cosine similarity (text-embedding-ada-002) with the original question. Context relevance: have the LLM extract the sentences from the context that are needed, score = extracted sentences / total sentences. Outputs are requested in JSON for more consistent scores. The library integrates with LlamaIndex and LangChain.
Dataset and evaluation	WikiEval, a new dataset of 50 Wikipedia pages describing events after the start of 2022, each with a ChatGPT-generated question, a context and answers. Two annotators judged pairs of answers or contexts on the three dimensions (agreement about 95 percent for faithfulness and context relevance, about 90 percent for answer relevance). Metrics are scored by how often they pick the same item as the humans (accuracy), against two baselines, GPT Score and GPT Ranking.
Key results	Agreement with humans (accuracy): faithfulness 0.95, answer relevance 0.78, context relevance 0.70 for RAGAs, against 0.72, 0.52 and 0.63 for GPT Score and 0.54, 0.40 and 0.52 for GPT Ranking. Context relevance was the hardest dimension, especially for long contexts.
Limitations	Depends on the LLM that does the scoring (the authors recommend open models over paid APIs); WikiEval is small and English-only; results can vary between runs and API versions.

Aspect	Summary
Relevance to the project	**High.** It gives the project a ready-made, cheap way to evaluate answer quality without writing reference answers for every Confluence page. Faithfulness maps to our "answers only from retrieved text" rule; context relevance helps tune chunk size and top-k. Cautions: the metrics were validated only on English, but the project must handle Vietnamese as well, so scores on Vietnamese questions should be spot-checked by hand; and the scoring LLM would have to run locally or on approved infrastructure if the questions touch internal data (inference). RAGAs measures answer quality only, so it must be combined with separate permission tests (see Section 3).

5. Architecture and Infrastructure

5.1 Boonsri & Buranasaksee (2025): A Repository-First Framework for Democratizing Chatbot Development

Aspect	Summary
Reference	Boonsri, R & Buranasaksee, U 2025, 2025 9th International Conference on Information Technology (InCIT), IEEE, pp. 775-781, doi:10.1109/INCIT66780.2025.11276047, Rajamangala University of Technology Suvarnabhumi, Thailand.
Purpose and topic	Hosted chatbot platforms are quick to set up but lock the customer in; custom frameworks give control but exclude non-developers. The paper tests whether a repository-first framework combined with an AI coding agent can cut development time and cost without lowering reliability or usability.

Aspect	Summary
Method	A four-layer Clean Architecture in C#/.NET: Presentation (ASP.NET Core APIs, admin UI, SignalR monitoring, signature checks and rate limits), Application (CQRS with MediatR, plug-in mechanism for new features), Domain (session state, message history, optional embeddings) and Infrastructure (adapters, PostgreSQL via EF Core with pgvector for semantic search, Redis for sessions and caching, LLM access, Google APIs). A domain expert fills a short playbook; the coding agent generates handlers, routes, validators and tests, which a reviewer accepts through an acceptance checklist; everything stays as plain Git files.
Dataset and evaluation	Within-subjects experiment: nine professional developers, eight matched microtasks on a LINE chatbot with Google Sheets, two conditions (Framework+Agent against Agent-only), 144 runs, 90-minute cap per condition, counterbalanced order. A separate System Usability Scale (SUS) study with 50 users (managers, project support, sales). The agent was run on a local model setup (ModelHarbor with a Qwen3 model) with Qdrant for code indexing.
Key results	Framework+Agent was faster and cheaper: time ratio (agent-only / framework) about 2.4 and cost ratio about 1.9, with 1.9 times fewer human interventions and similar error rates. Quality: normalised QA accuracy 0.93. SUS mean 70.6 (above the common benchmark of 68). Six of eight tasks favoured the framework; the two simplest tasks favoured agent-only.

Aspect	Summary
Limitations	Only eight paired tasks, so the study is under-powered (the authors say so); one technology stack; participants were professional developers; no production data. The reported figures are not fully consistent inside the paper (for example the time ratio appears as 2.29 in the abstract, 2.44 in Table I and 2.48 in the text, and the SUS confidence interval differs between sections), so cite the abstract values or round figures and do not over-quote decimals.
Relevance to the project	**Medium.** Its layered architecture (presentation, application, domain, infrastructure) is close to our four layers, and it independently chooses PostgreSQL with pgvector and a cache, which supports our storage decision. It also uses the SUS questionnaire, which we can reuse for the client's usability test. It is about building chatbots with a coding agent, not about retrieval or security, so it supports architecture and process choices and not the research problem.

5.2 Mohammed & Aref (n.d.): Chatbot System Architecture

Aspect	Summary
Reference	Mohammed, M & Aref, MM n.d., 'Chatbot System Architecture', Faculty of Computer and Information Sciences, Ain Shams University, Cairo. **The year and venue are not stated in the copy reviewed; confirm before citing.**
Purpose and topic	A descriptive overview of the standard architecture of a conversational agent, explaining each component.

Aspect	Summary
Method	Literature-based description (no experiment). Three core components: Natural Language Understanding (topic detection, intent analysis, entity linking), Dialogue Manager (rule-based, knowledge-based, online information retrieval, neural reply generation, context tracking) and Natural Language Generator (content filter, engagement ranking, text or speech output). A case study of the KLM airline Messenger bot.
Dataset and evaluation	None. The KLM case study reports, from a customer survey, higher interaction through Messenger (stated as 40 percent) and about 15 percent success of online boarding bookings credited to the bot, without a method or baseline.
Key results	No measured results. The conclusion is that the Dialogue Manager is the most important component because its parts can be improved or replaced.
Limitations	Pre-LLM view of chatbots (rules, intents, neural generation); no retrieval-augmented generation; no security or access control; informal language and unclear sources for the case study figures.
Relevance to the project	**Low.** Useful only as a short textbook-level reference for how a chatbot is conventionally split into understanding, dialogue management and generation. In our system the router and the LLM replace most of these components, so the paper helps to explain terminology to a non-technical client but should not be used as evidence for design choices.

5.3 Malkov & Yashunin (2020): Efficient and Robust Approximate Nearest Neighbor Search Using HNSW Graphs

Aspect	Summary
Reference	Malkov, YA & Yashunin, DA 2020, IEEE Transactions on Pattern Analysis and Machine Intelligence (the copy reviewed shows a template header; **confirm volume and pages before citing**; arXiv:1603.09320).
Purpose and topic	Finding the K nearest vectors to a query quickly in large collections (approximate K-nearest-neighbour search), which is the core operation of vector search in RAG.
Method	Hierarchical Navigable Small World (HNSW): a multi-layer graph. Each element is assigned a maximum layer at random with an exponentially decaying probability, so upper layers are sparse and give long links, lower layers are dense. A search enters at the top layer and moves greedily down; a heuristic for choosing neighbours improves recall on clustered data. Main parameters are M (links per element), efConstruction (candidate list while building) and ef (candidate list while searching). Search cost scales roughly logarithmically with the data size.
Dataset and evaluation	Implemented in C++ in nmslib. Compared with baseline NSW, FLANN, Annoy, VP-tree, FALCONN and others on the ann-benchmarks vector sets (SIFT 1M, GloVe 1.2M, CoPhIR 2M, DEEP 1M, MNIST 60k and random vectors up to 30M), on non-metric sets (Wiki-sparse, Wiki-8/128, ImageNet signatures, DNA) and against product-quantisation methods on a 200M SIFT set.
Key results	HNSW outperforms the other open-source vector-only methods in the reported tests, uses far fewer distance computations than NSW, and shows near-logarithmic scaling with data size.

Aspect	Summary
Limitations	The authors list as future work support for element updates and removal, and note that, unlike basic NSW, distributed search is not straightforward. Memory use is higher than compressed (quantised) methods.
Relevance to the project	**Medium.** pgvector's HNSW index implements this algorithm, so the paper is the correct citation for why our search stays fast as Confluence grows, and for the M and ef parameters we will tune. The point on updates and removal links to a known issue in our design (inference): when a Confluence page is edited or deleted, old chunks must be replaced or removed, which the original algorithm does not address; this is why the periodic clean-up job is proposed.

6. Synthesis, Gaps and Points for the Presentation

6.1 What the ten papers show together

1. **Access control is the gap in most of the literature.** Only Jeong & Lee (2025) and Bhatt et al. (2025) deal with who may see what in a RAG system, and Zhou et al. (2025) offers an encryption design without experiments. Lewis et al. (2020) and Gao et al. (2024) do not cover access control at all. This supports the project's claim that permission-aware retrieval is the distinguishing requirement.
2. **Our design differs from the closest work in two ways.** Jeong & Lee retrieve the top-k first and then check each document with the owner's IAM system, so answer quality drops when a user may read few documents. The project applies the permission filter inside the database query, so the top-k is chosen only from permitted text. The cost (inference) is that permissions are copied at ingestion time and may be out of date until the next sync, whereas Jeong & Lee check in real time. This trade-off should be stated openly.
3. **Slack adds a participant problem.** Bhatt et al. argue that content must be authorised for all participants. A Slack thread or channel can be read by every member, so the bot's answer should not exceed what the audience may see (inference). This is not yet decided in our design and is a good open question to raise.

4. **Evaluation has two separate parts.** Answer quality can be measured with RAGAs (faithfulness, answer relevance, context relevance) and the Gao et al. vocabulary; permission correctness needs its own tests, such as the permission-coverage, exact-match and F1 measures used by Jeong & Lee, and negative-rejection tests. RAGAs was validated only on English, so Vietnamese results need manual checking.
5. **Infrastructure choices are supported but not proven for our case.** Boonsri & Buranasaksee (2025) use PostgreSQL with pgvector and a cache in a layered architecture, and Malkov & Yashunin (2020) is the algorithm behind the pgvector HNSW index. The original HNSW paper lists updates and removal as future work, which relates to our need to replace or remove chunks when Confluence pages change.
6. **Source quality differs.** Peer-reviewed or published venues: Lewis et al. (NeurIPS), Jeong & Lee (IEEE Access), Es et al. (EACL), Boonsri & Buranasaksee (IEEE InCIT), Mukherjee et al. (ACM FSE Companion) and Malkov & Yashunin (IEEE TPAMI). Preprints: Gao et al., Bhatt et al. and Zhou et al. (arXiv). Mohammed & Aref has no venue in the copy reviewed. Say "preprint" when citing the arXiv papers.

6.2 Gaps the project can claim to address

- Permission-aware retrieval for a Slack and Confluence setting, with the filter inside the vector query rather than after retrieval.
- A bilingual (English and Vietnamese) enterprise knowledge assistant; none of the ten papers evaluates Vietnamese.
- Evaluation that reports answer quality and permission correctness together.

6.3 Limits to state honestly

- Zhou et al. has no experiments, and Bhatt et al. has no measured results; both support the argument, not a quantitative claim.
- The Jeong & Lee experiments use HotpotQA, not enterprise data, and Mukherjee et al. use 100 questions in one domain, so numbers from these papers should not be presented as expected results for our system.
- Items marked as inference are the author's reading and should be introduced as such when presenting.

6.4 Suggested order and one-line message per paper for the presentation

1. Lewis et al.: this is the pattern we implement (retrieve, then generate, with updatable knowledge).
2. Gao et al.: this is how we describe and evaluate our version of the pattern.
3. Jeong & Lee: this is the closest work on permissions, and where our design differs.

4. Bhatt et al.: this is why permissions must be enforced outside the model, and the participant question for Slack.
5. Es et al. (RAGAs): this is how we will measure answer quality.
6. Mukherjee et al. and Boonsri & Buranasaksee: examples of enterprise assistants and layered architecture.
7. Malkov & Yashunin: why vector search stays fast; the update and delete caveat.
8. Zhou et al. and Mohammed & Aref: contrast and terminology only.

6.5 Questions a strict lecturer may ask

- Why filter before retrieval and not after? (Answer: avoids quality loss at low permission coverage; trade-off is stale permissions between syncs.)
- How do you know the answers are correct and permitted? (Answer: RAGAs for quality, separate permission tests for access.)
- Who can see the bot's answer in a channel? (Answer: open decision; Bhatt et al. suggest limiting to what all participants may see.)
- Why trust arXiv preprints? (Answer: they are used for argument and vocabulary, and the claims that carry weight come from published venues.)
- Does it work in Vietnamese? (Answer: not validated by the papers; we test it ourselves.)

7. References

Bhatt, SS et al. 2025, *Enterprise AI must enforce participant-aware access control*, arXiv:2509.14608, preprint.

Boonsri, R & Buranasaksee, U 2025, 'A repository-first framework for democratizing chatbot development', *2025 9th International Conference on Information Technology (InCIT)*, IEEE, pp. 775-781, doi:10.1109/INCIT66780.2025.11276047.

Es, S, James, J, Espinosa-Anke, L & Schockaert, S 2024, 'RAGAs: automated evaluation of retrieval augmented generation', *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, Association for Computational Linguistics, pp. 150-158.

Gao, Y et al. 2024, *Retrieval-augmented generation for large language models: a survey*, arXiv:2312.10997, preprint.

Jeong, J & Lee, S-G 2025, 'Permission-aware retrieval-augmented generation', *IEEE Access*, vol. 13, pp. 192819-192835, doi:10.1109/ACCESS.2025.3628960.

Lewis, P et al. 2020, 'Retrieval-augmented generation for knowledge-intensive NLP tasks', *Advances in Neural Information Processing Systems*, vol. 33.

Malkov, YA & Yashunin, DA 2020, 'Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, arXiv:1603.09320 (confirm volume and pages before citing).

Mohammed, M & Aref, MM n.d., *Chatbot system architecture*, Ain Shams University, Cairo (confirm year and venue before citing).

Mukherjee, M, Kim, S, Chen, X, Luo, D, Yu, T & Mai, T 2025, 'From documents to dialogue: building KG-RAG enhanced AI assistants', *Companion Proceedings of the 33rd ACM Symposium on the Foundations of Software Engineering (FSE '25)*, ACM, arXiv:2502.15237.

Zhou, P, Feng, Y & Yang, Z 2025, *Privacy-aware RAG: secure and isolated knowledge retrieval*, arXiv:2503.15548, preprint.